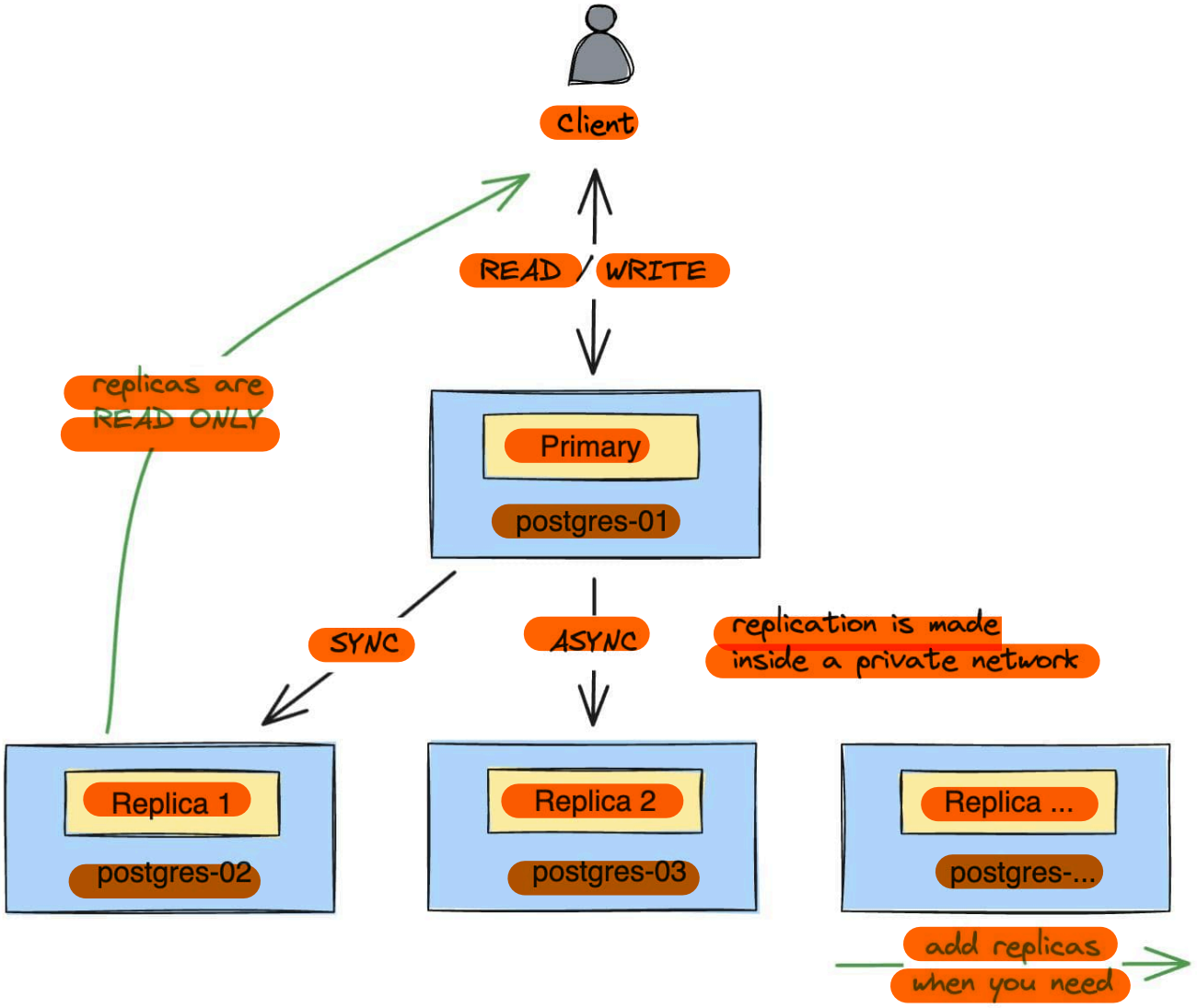
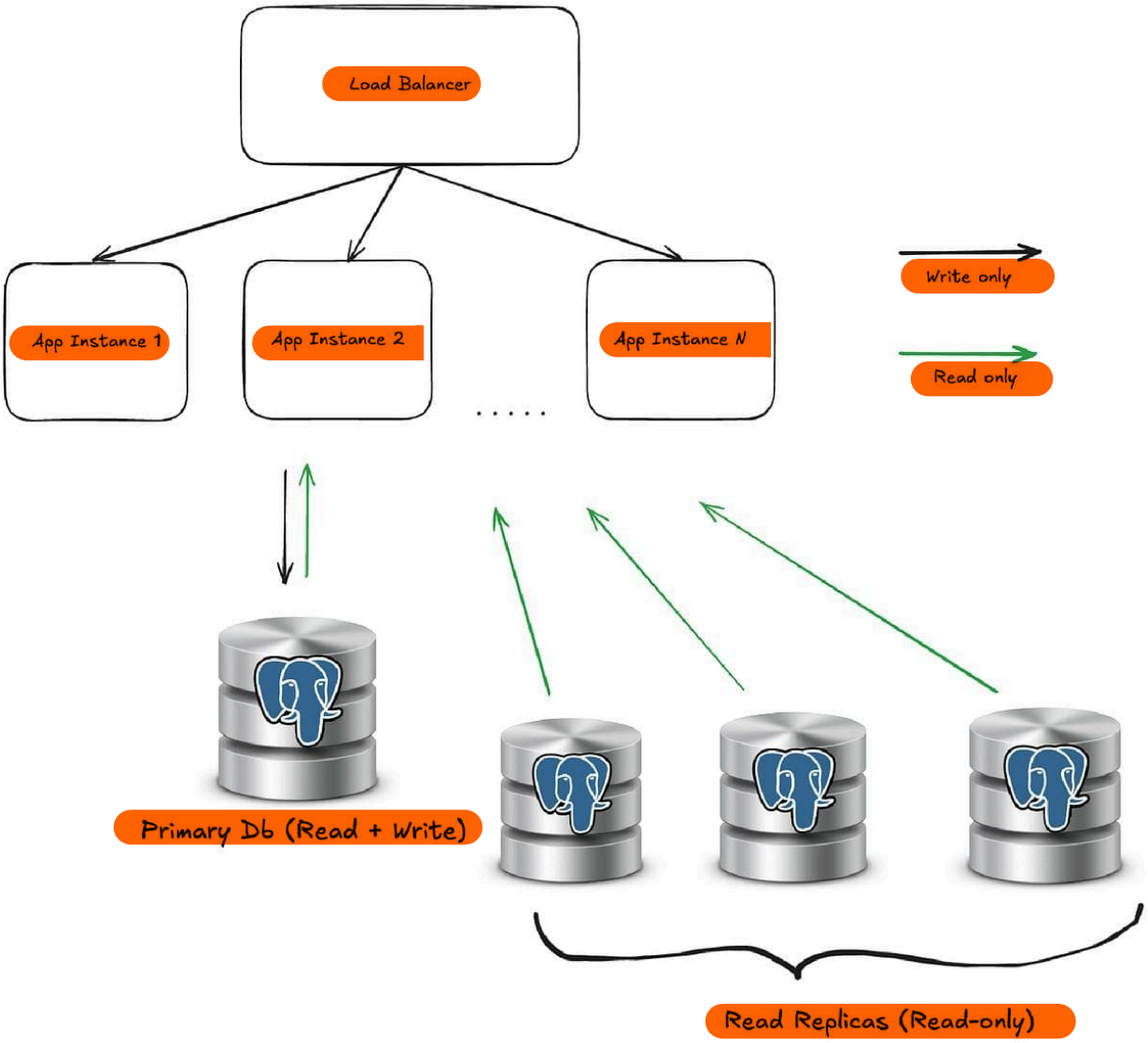
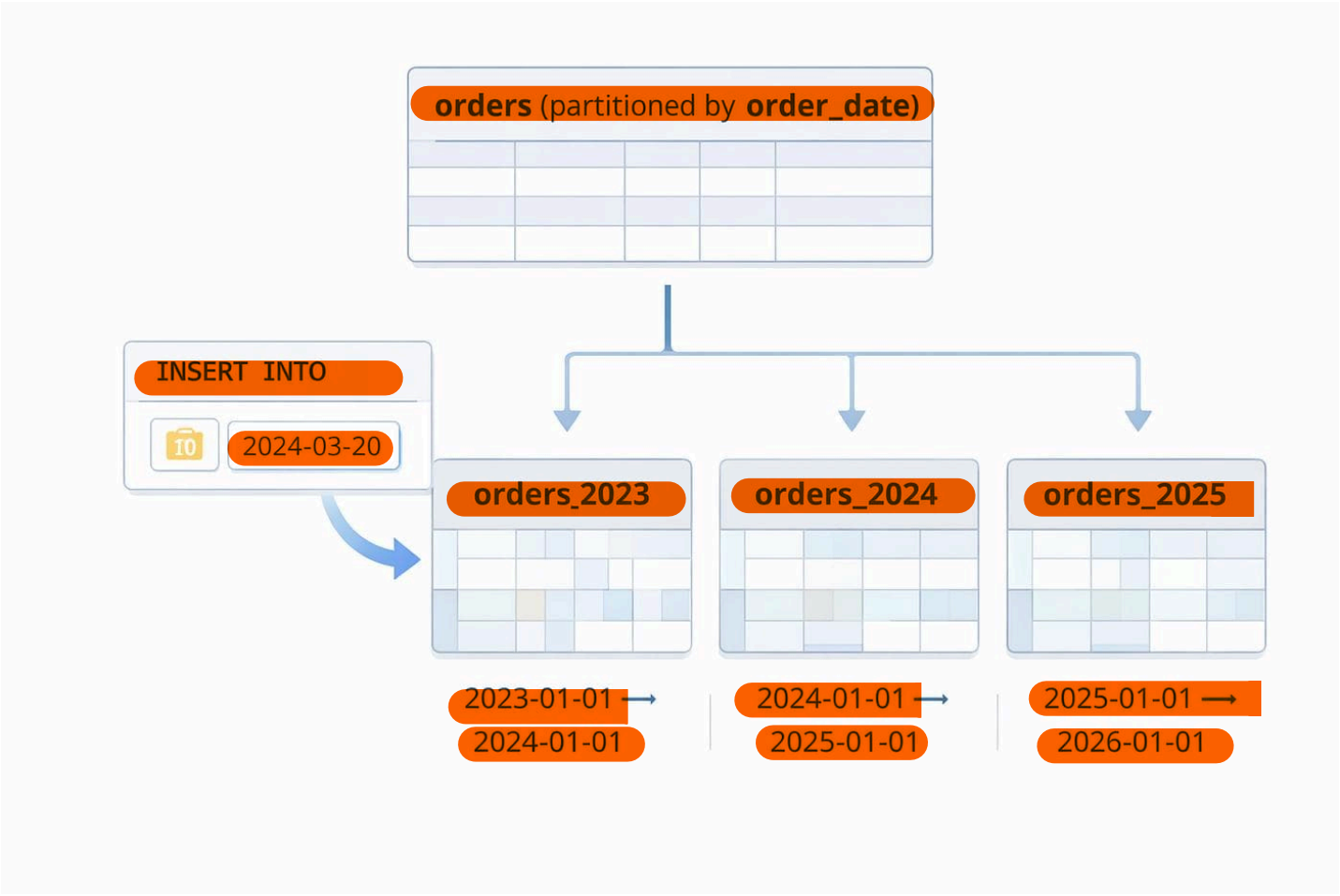
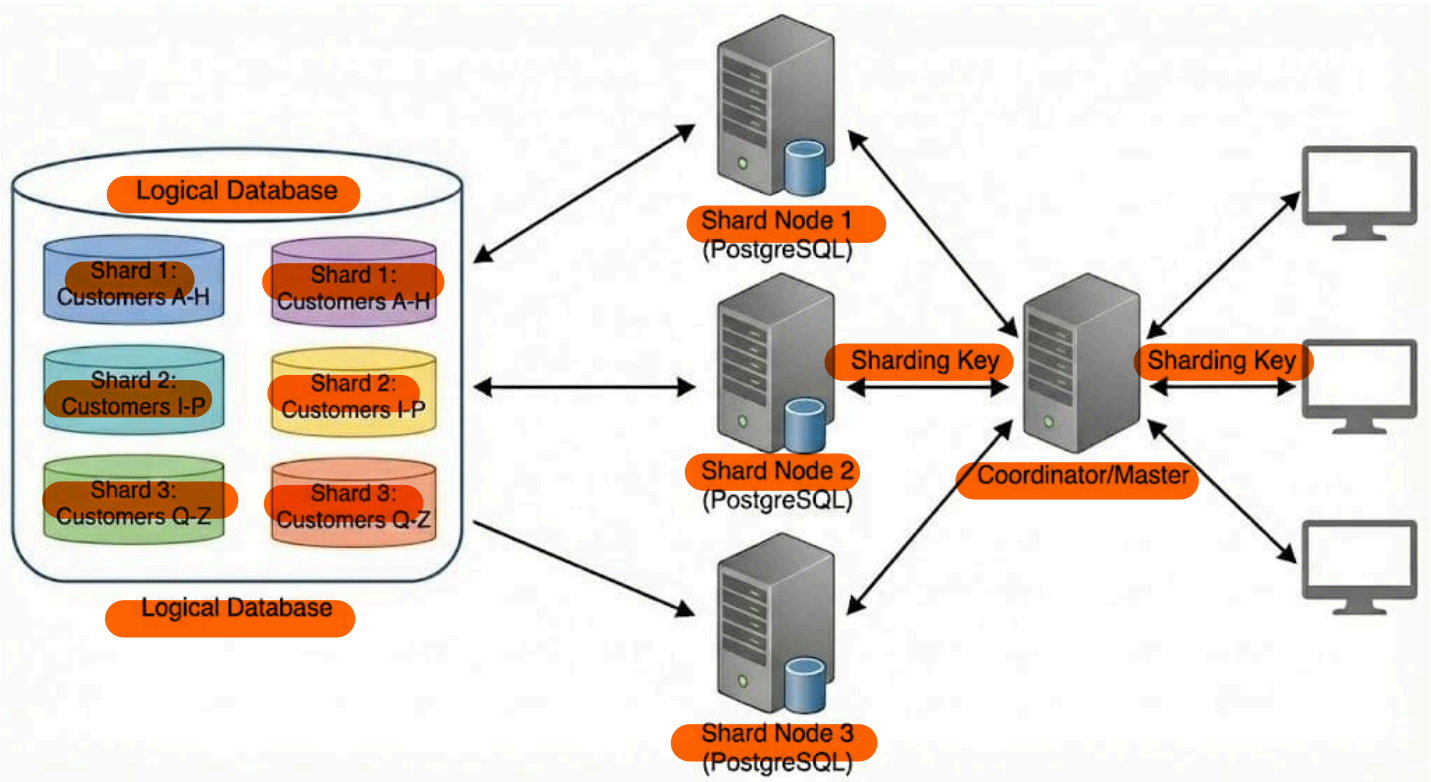
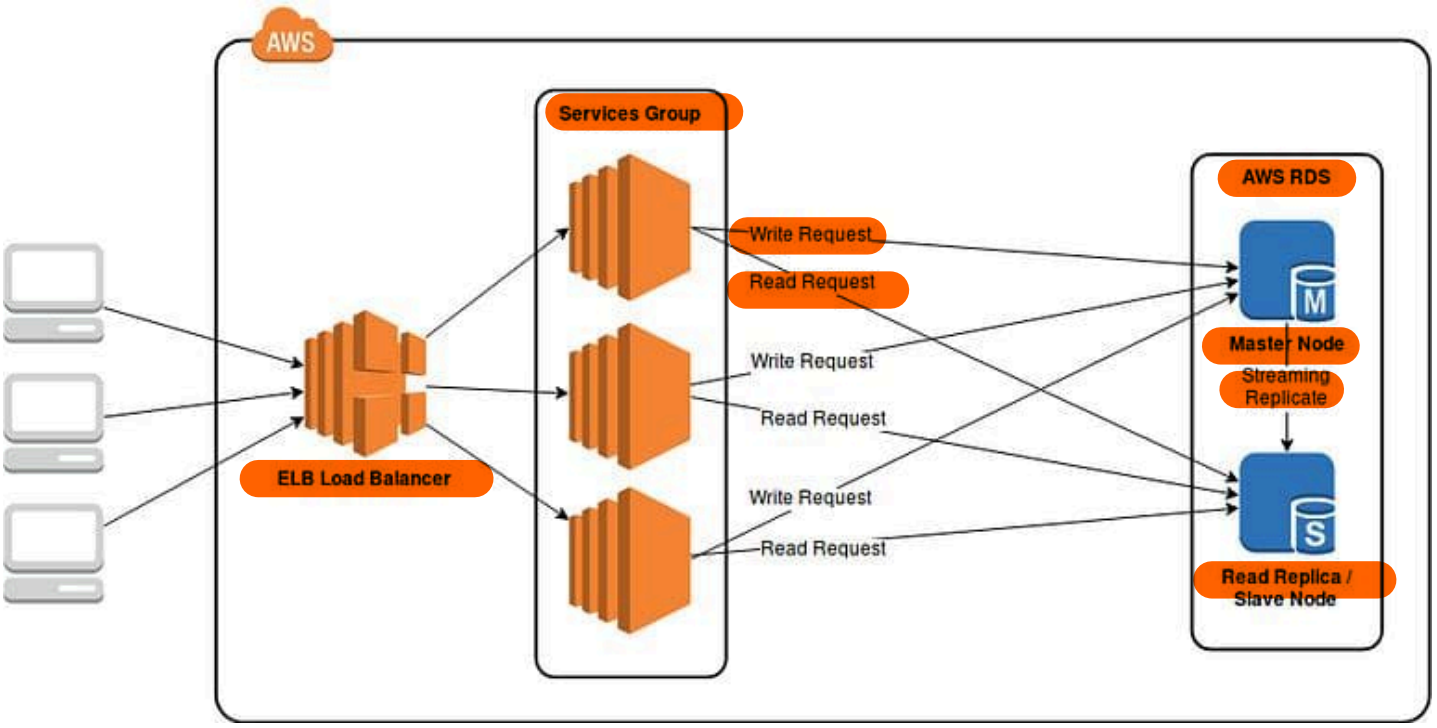
From a **system-design interview** perspective, scaling PostgreSQL is best understood as a progression from **vertical scaling** → **connection/read scaling** → **partitioning** → **sharding**, while preserving PostgreSQL as the system of record as long as practical.

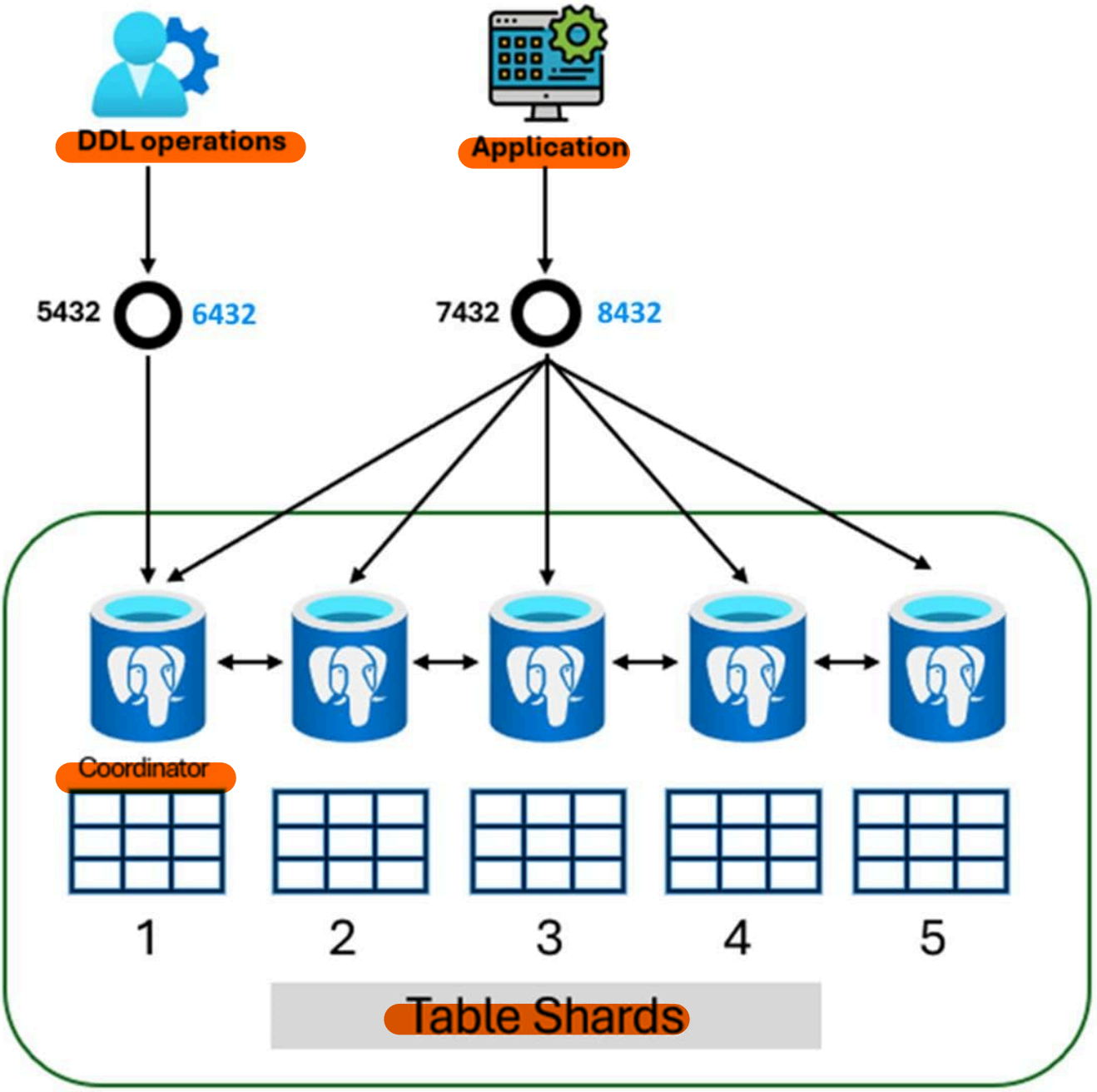



Load Balancer
App Instance 1
App Instance 2
App Instance N
Write only
Read only
Primary Db (Read + Write)
Read Replicas (Read-only)


orders (partitioned by order_date)
INSERT INTO
2024-03-20
orders_2023
orders_2024
orders_2025
2023-01-01
2024-01-01
2024-01-01
2025-01-01
2025-01-01
2026-01-01


Logical Database
Shard 1: Customers A-H
Shard 1: Customers A-H
Shard 2: Customers I-P
Shard 2: Customers I-P
Shard 3: Customers Q-Z
Shard 3: Customers Q-Z
Logical Database
Shard Node 1 (PostgreSQL)
Shard Node 2 (PostgreSQL)
Shard Node 3 (PostgreSQL)
Sharding Key
Sharding Key
Coordinator/Master


AWS
Services Group
AWS RDS
Write Request
Read Request
Write Request
Read Request
Write Request
Read Request
Master Node
Streaming Replicate
ELB Load Balancer
Read Replica / Slave Node



1. Start with the scaling dimensions

Before choosing an architecture, identify the bottleneck:

Bottleneck	Typical solution
CPU	Larger instance, query optimization, replicas
RAM	Larger instance, indexes, `shared_buffers`, working-set optimization
Disk I/O	Faster SSD/NVMe, WAL optimization, partitioning
Storage capacity	Larger disks, partitioning, archival

Bottleneck	Typical solution
Connections	Connection pooling
Read QPS	Read replicas
Write QPS	Query/schema optimization → partitioning → sharding
Large tables	Partitioning
HA requirements	Streaming replication + automatic failover
Geographic latency	Regional replicas / distributed architecture
Extremely high scale	Sharding / distributed PostgreSQL

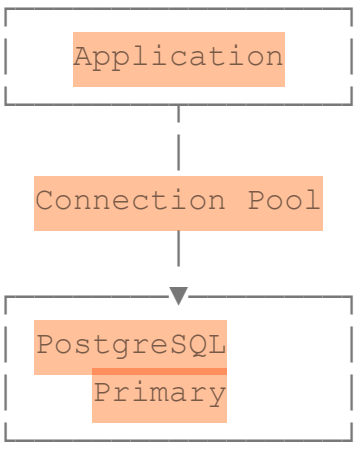
The important interview principle is:

> **Don't introduce sharding merely because the database is "large." Identify the actual bottleneck first.**

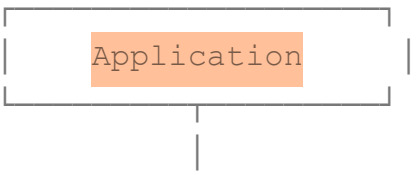
---

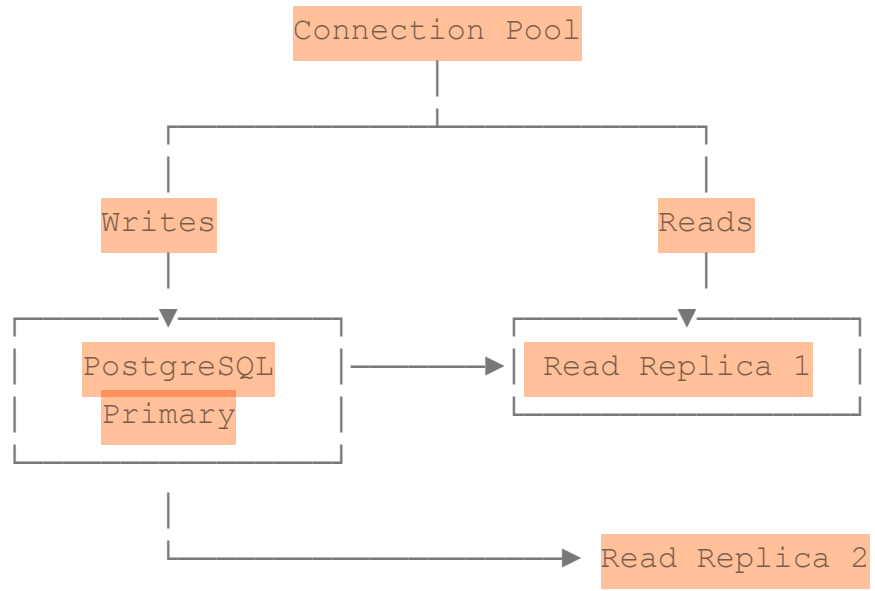
2. Scaling PostgreSQL progressively

A mature architecture usually evolves roughly like this:

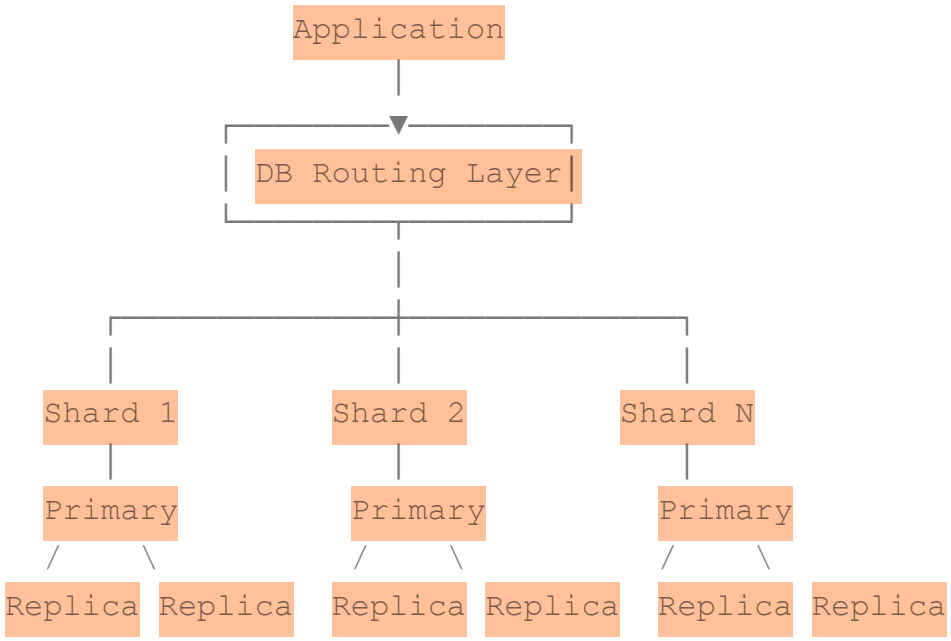


Then:





Eventually:



Each step solves a different class of problem.

---

3. Step 1 — Vertical scaling

The simplest and often most effective optimization is a larger PostgreSQL server.

Increase:

- CPU
- RAM
- NVMe/SSD IOPS
- network bandwidth

- storage capacity

For example:

```
8 CPU / 32 GB RAM
        ↓
32 CPU / 128 GB RAM
        ↓
64 CPU / 256 GB RAM
```

This can dramatically increase capacity without introducing distributed-system complexity.

Why vertical scaling works

A database benefits heavily from memory because:

```
Application query
        |
        ▼
PostgreSQL
        |
        ├── shared buffers
        |
        ├── OS page cache
        |
        └── disk
```

If the working set fits in memory, many reads avoid disk I/O.

Interview point

Vertical scaling should usually be exhausted before sharding.

---

4. Step 2 — Optimize queries and indexes

Before adding infrastructure, optimize the workload.

Typical techniques:

Indexing

```
CREATE INDEX idx_orders_user_id
ON orders(user_id);
```

Composite indexes:

```
CREATE INDEX idx_orders_user_status_created
ON orders(user_id, status, created_at);
```

Partial indexes:

```
CREATE INDEX idx_active_orders
ON orders(user_id)
WHERE status = 'ACTIVE';
```

Covering indexes:

```
CREATE INDEX idx_orders_user
ON orders(user_id)
INCLUDE (status, created_at);
```

---

5. Use `EXPLAIN ANALYZE`

A system-design answer should mention query plans.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM orders
WHERE user_id = 123;
```

Look for:

- sequential scans
- inefficient joins
- incorrect cardinality estimates
- excessive disk reads

- sorts
- hash operations
- poor index utilization

The principle:

```
Bad query
    ↓
More CPU / I/O
    ↓
Less throughput
    ↓
Add replicas
    ↓
More infrastructure
    ↓
Still bad query
```

So **query optimization precedes horizontal scaling**.

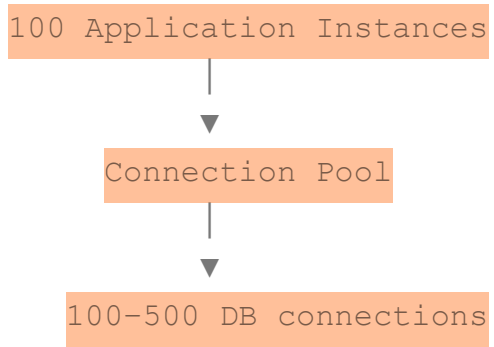
---

6. Step 3 — Connection pooling

PostgreSQL uses a process/thread architecture where maintaining huge numbers of connections is expensive.

Don't let every application instance create hundreds of independent PostgreSQL connections.

Instead:

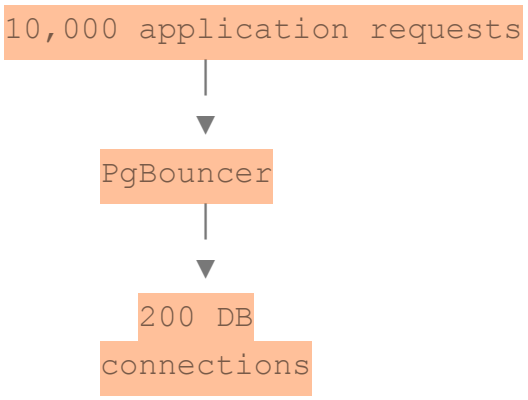


A common choice is PgBouncer.

Connection pooling provides:

- connection reuse
- lower connection overhead
- protection against connection storms
- better database resource utilization

For example:

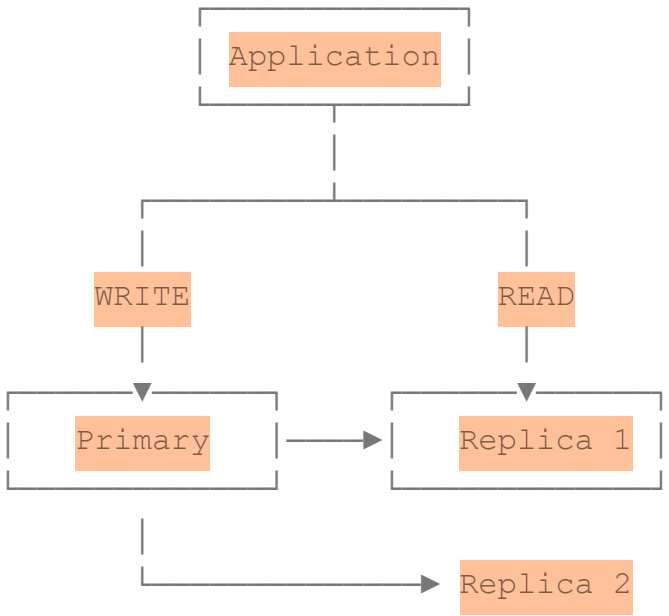


This can be one of the highest-impact scaling improvements.

---

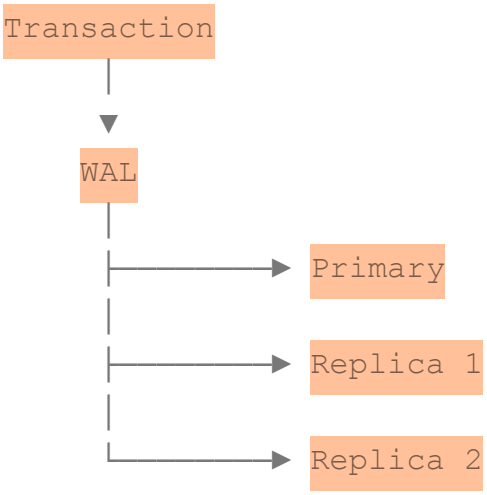
7. Step 4 — Read replicas

If the workload is read-heavy, introduce streaming replication.



PostgreSQL uses **WAL — Write-Ahead Logging**.

Conceptually:

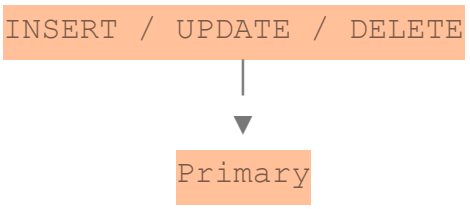


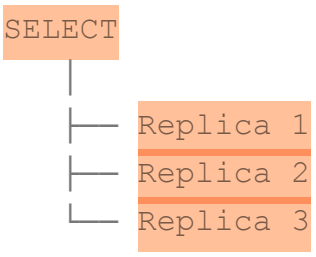
Replicas replay WAL to reconstruct the primary's state.

---

8. Read/write splitting

Your application can route:





This increases **read throughput**, but does not fundamentally increase write throughput.

That distinction is extremely important in interviews.

Example

Suppose:

```
100,000 QPS
90,000 reads
```

```
10,000 writes
```

Adding replicas can help substantially.

But if:

```
90,000 writes
10,000 reads
```

replicas don't solve the primary write bottleneck.

---

9. Replica consistency

Read replicas introduce an important problem:

> **Replication lag.**

Consider:

```
t0:
Primary: balance = $100
Replica: balance = $100

t1:
UPDATE balance = $50

t2:
Primary: $50
Replica: $100
```

A subsequent read from the replica can return stale data.

Therefore, system design must explicitly decide which reads tolerate eventual consistency.

Pattern: read-your-writes

After:

```
POST /order
```

route subsequent reads temporarily to the primary.

Or use:

```
session → primary
```

for consistency-sensitive workflows.

Meanwhile:

```
analytics
search
recommendations
dashboards
```

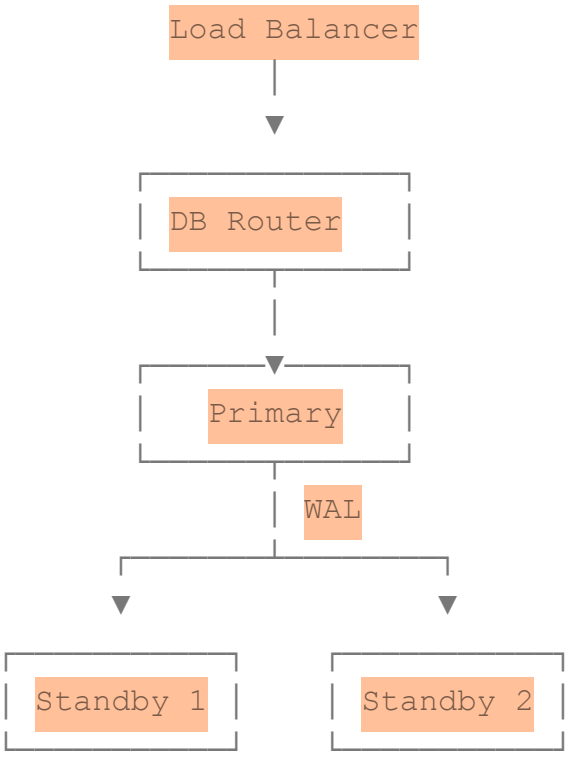
can often use replicas.

---

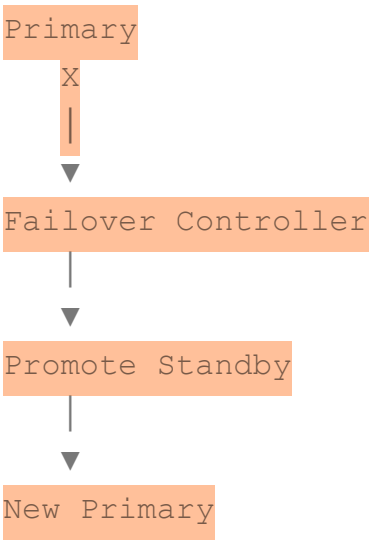
10. Step 5 — High availability

Now separate **scaling** from **availability**.

A typical HA architecture:



If primary fails:



Common components/ecosystems include:

- PostgreSQL streaming replication
- Patroni
- etcd/Consul
- HAProxy
- PgBouncer
- Kubernetes operators
- managed PostgreSQL services

The key distinction:

Replication ≠ automatic failover.

Replication copies data.

A failover system determines:

1. Is the primary actually dead?
2. Which standby is safest?
3. Who should become primary?
4. How do clients discover the new primary?
5. How do we prevent split-brain?

---

11. Step 6 — Partitioning

When individual tables become enormous, PostgreSQL declarative partitioning becomes important.

Suppose:

```
orders
---------
10 billion rows
```

Partition by time:

```
orders
  ├── orders_2026_01
  ├── orders_2026_02
  ├── orders_2026_03
  ├── ...
  └── orders_2026_12
```

Example:

```
CREATE TABLE orders (
    id BIGINT,
    created_at TIMESTAMP NOT NULL,
    user_id BIGINT,
    amount NUMERIC
) PARTITION BY RANGE (created_at);
```

Then:

```
CREATE TABLE orders_2026_01
PARTITION OF orders
FOR VALUES FROM ('2026-01-01')
             TO ('2026-02-01');
```

---

12. Why partitioning helps

Suppose the query is:

```
SELECT *
FROM orders
```

```
WHERE created_at >= '2026-09-01'
  AND created_at < '2026-10-01';
```

PostgreSQL can perform **partition pruning**:

```
orders
 │
 ├── Jan ── skip
 ├── Feb ── skip
 ├── ...
 ├── Aug ── skip
 ├── Sep ── READ
 ├── Oct ── skip
 └── ...
```

Instead of scanning the entire logical table.

Partitioning also helps operationally:

- archive old data
- drop old partitions quickly
- maintain indexes independently
- reduce vacuum burden
- manage lifecycle policies

---

13. Partitioning strategies

Range partitioning

Best for:

```
timestamp
date
sequence
numeric ranges
```

Example:

```
2026-01
2026-02
2026-03
```

List partitioning

Useful for:

```
country
tenant type
region
category
```

Example:

```
US
IN
UK
DE
```

Hash partitioning

Useful when you want relatively uniform distribution:

```
hash(user_id) % 16
```

```
partition 0
partition 1
...
partition 15
```

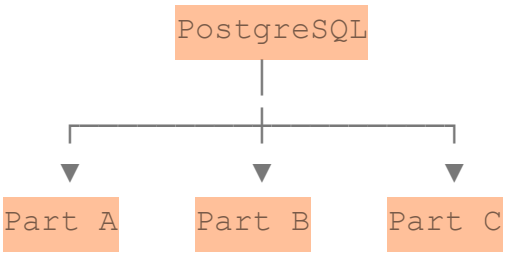
---

14. Partitioning ≠ sharding

This distinction is frequently tested.

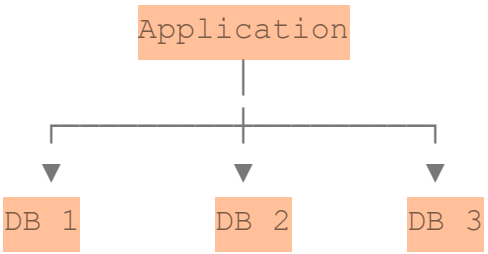
Partitioning

Data is divided **inside one PostgreSQL database**.



Sharding

Data is distributed across **multiple PostgreSQL instances**.



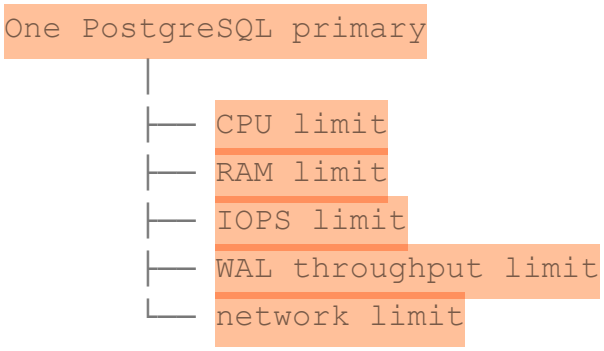
Partitioning primarily helps manage large tables.

Sharding primarily helps overcome the capacity limits of one database instance.

---

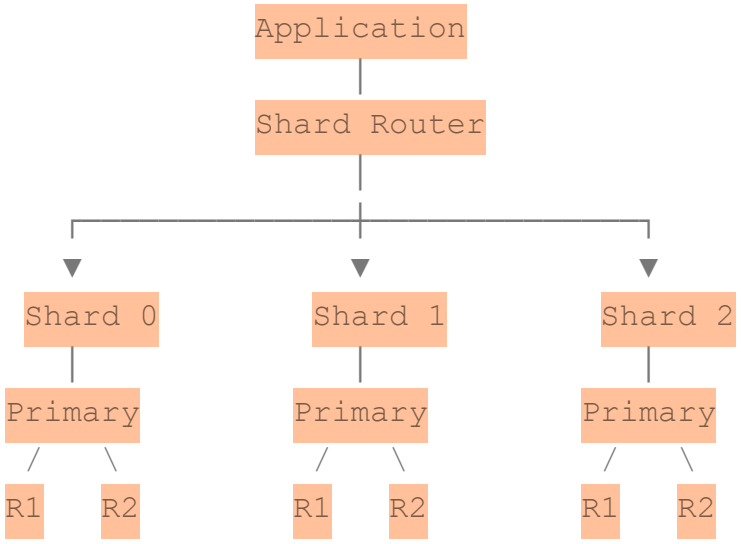
15. Step 7 — Sharding

Eventually you may reach the fundamental limit:



At this point, distribute writes across multiple PostgreSQL databases.

Example:



---

16. Choosing a shard key

This is probably the most important sharding decision.

Suppose we have:

```
users
orders
payments
```

A natural shard key might be:

```
user_id
```

Then:

```
hash(user_id) % N
```

determines the shard.

Example:

```
user 101 → shard 3
user 102 → shard 1
user 103 → shard 7
```

The shard key should ideally provide:

1. Even distribution
2. High cardinality
3. Stable assignment
4. Query locality
5. Minimal cross-shard operations

---

17. Bad shard keys

Suppose you shard by:

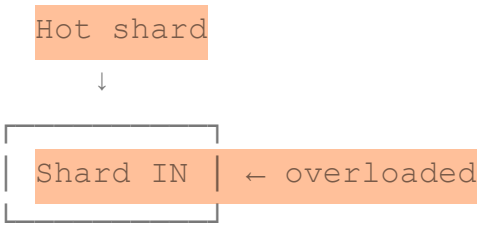
```
country
```

and 50% of users are from India.

You get:

```
India → 50% traffic
US    → 15%
UK    → 5%
...
```

Result:




```
Other shards → underutilized
```

This is a **hotspot**.

---

18. Hash vs range sharding

Hash sharding

```
hash(user_id) → shard
```

Advantages:

- good distribution
- minimizes hotspots

Disadvantages:

- range queries are difficult

For example:

```
WHERE user_id BETWEEN 100000 AND 200000
```

may hit many shards.

Range sharding

```
1–1M        → shard 1
1M–2M       → shard 2
2M–3M       → shard 3
```

Advantages:

- efficient range queries
- easier locality

Disadvantages:

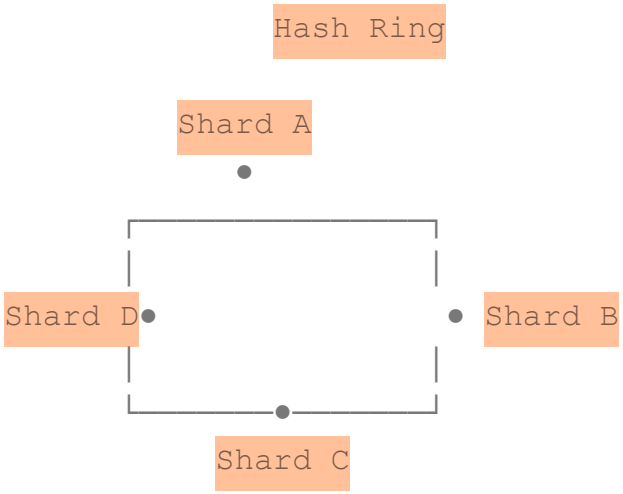
- hotspots
- uneven distribution
- difficult resharding

---

19. Consistent hashing

For dynamic environments, consistent hashing can reduce data movement when adding/removing shards.

Conceptually:



But don't automatically assume consistent hashing is the best answer.

Many production systems use:

```
logical shards
      ↓
physical PostgreSQL nodes
```

For example:

```
10,000 logical shards
        |
        ▼
    20 servers
```

This makes rebalancing much easier.

---

20. Logical shards are extremely useful

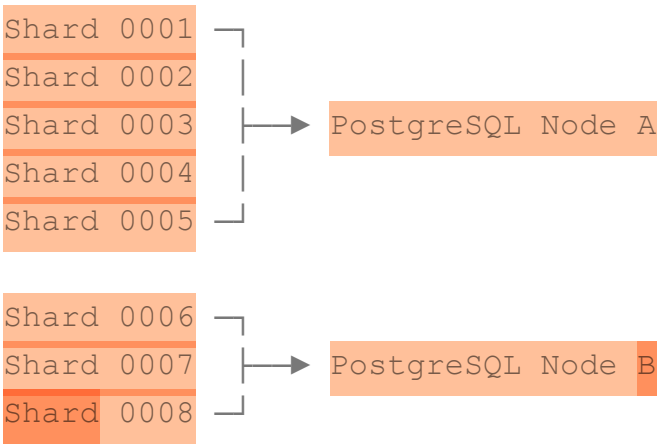
Instead of:

```
user → physical server
```

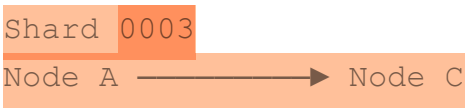
use:



For example:



Now you can move:



without changing the application's shard-key algorithm.

This is a strong production-grade design.

---

21. Cross-shard queries

Sharding introduces a major problem.

Suppose:

```
SELECT *
FROM orders
```

```
WHERE user_id = ?;
```

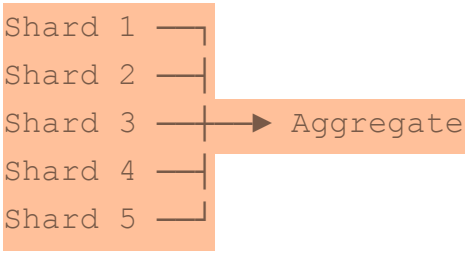
Easy:

```
user_id
   ↓
Shard router
   ↓
One shard
```

But:

```
SELECT SUM(amount)
FROM orders
WHERE created_at >= ...
```

might require:



This is a **scatter-gather query**.

It introduces:

- network overhead
- fan-out
- coordination
- partial failures
- latency amplification

Therefore:

> **Choose the shard key around the application's dominant access patterns, not merely around the database schema.**

---

22. Cross-shard transactions

This is another major difficulty.

Suppose:

```
Payment → Shard A
Order   → Shard B
```

and you need:

```
BEGIN

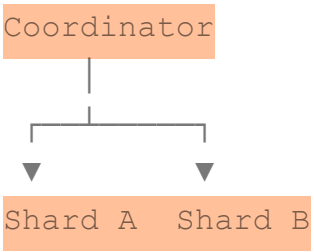
create payment
create order

COMMIT
```

A single PostgreSQL transaction cannot naturally span independent PostgreSQL instances.

Possible approaches include:

2-phase commit



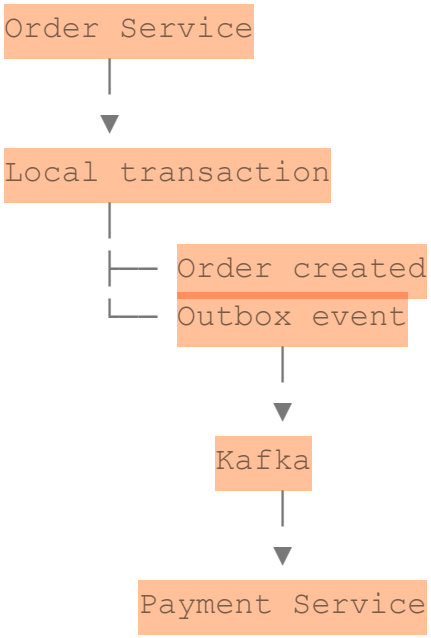
But distributed transactions introduce significant complexity and failure modes.

Better application patterns

Often prefer:

- Saga
- transactional outbox
- idempotency
- asynchronous events
- compensating transactions

Example:



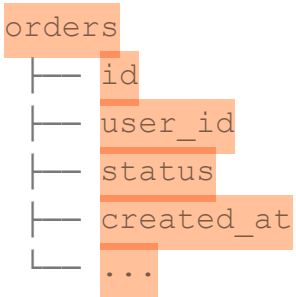
This keeps transactions local to a shard/service boundary.

---

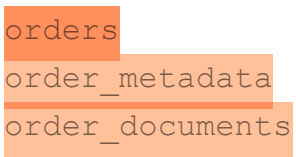
23. Schema design becomes critical

Avoid unnecessarily large rows.

For example:



Separate large rarely accessed data:



This reduces:

- I/O
- cache pressure
- index size

- vacuum overhead

---

24. Avoid unnecessary indexes

Indexes accelerate reads but make writes more expensive.

Every:

```
INSERT
UPDATE
DELETE
```

may need to update multiple indexes.

Suppose:

```
Table
  ├── PK index
  ├── user_id index
  ├── status index
  ├── created_at index
  ├── email index
  └── composite index
```

Write amplification can become significant.

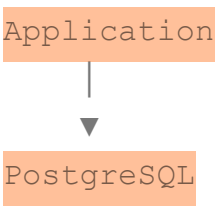
Therefore:

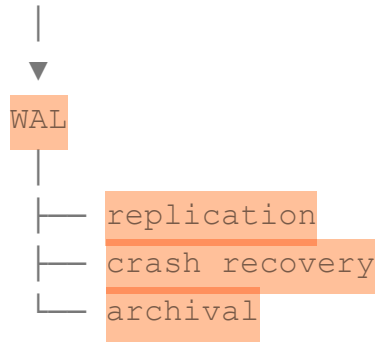
> Index according to actual query patterns, not every column that might someday be queried.

---

25. WAL becomes important at high scale

At high write rates:





The WAL subsystem can become a bottleneck.

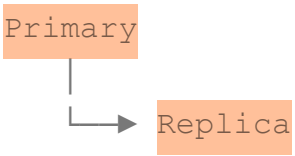
Things to consider:

- WAL volume
- WAL disk throughput
- replication bandwidth
- checkpoint behavior
- replication lag
- synchronous vs asynchronous replication

---

26. Synchronous vs asynchronous replication

Asynchronous



Primary doesn't wait for replica acknowledgment.

Advantages:

- lower write latency
- higher throughput

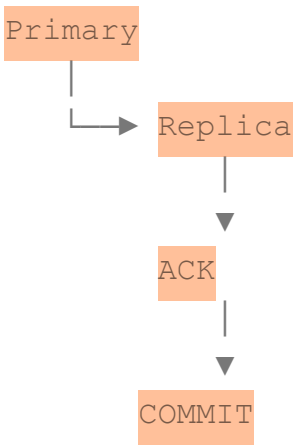
Risk:



```
      ↓
Potential data loss
```

This is measured using **RPO**.

Synchronous



Lower potential data loss, but higher latency and lower availability if synchronous replicas are unavailable.

---

27. Storage scaling

PostgreSQL data is ultimately constrained by storage.

Typical hierarchy:

```
HDD
 ↓
SSD
 ↓
NVMe
```

For latency-sensitive workloads:

```
NVMe
+
sufficient RAM
+
```

```
good indexes
+
query optimization
```

can be dramatically better than slow disks.

Also consider:

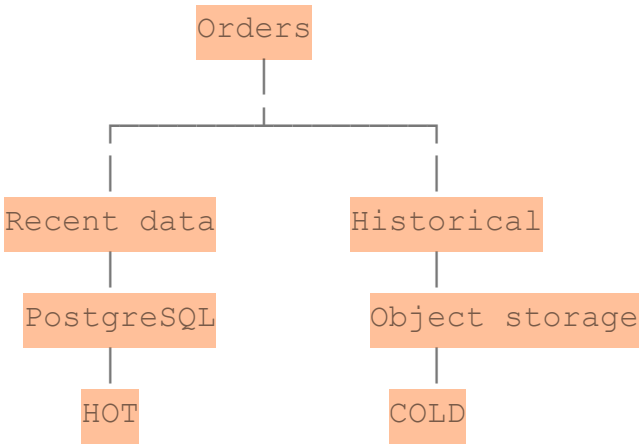
- tablespaces
- WAL placement
- storage autoscaling
- archival
- backups
- compression at application/storage layers

---

28. Archival strategy

Don't keep every historical record in the hot database indefinitely.

For example:



Example:

```
0–12 months → PostgreSQL
12+ months  → object storage / warehouse
```

Depending on requirements.

This dramatically reduces:

- table size

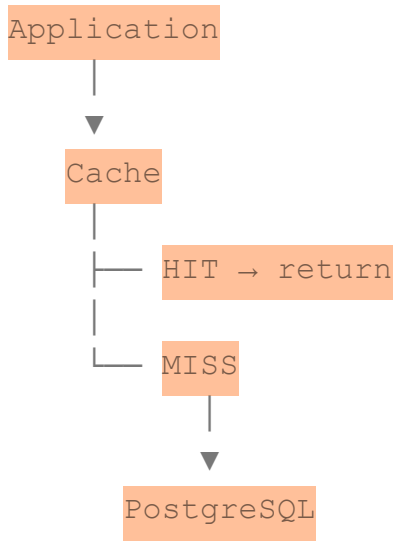
- index size
- vacuum workload
- backup size
- query cost

---

29. Caching

PostgreSQL shouldn't necessarily serve every read.

Introduce a cache for highly repeated data:

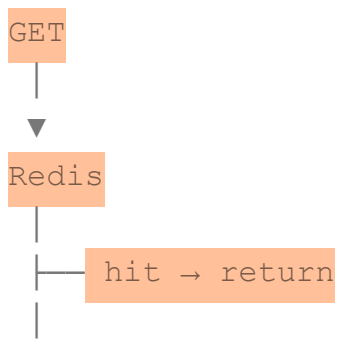


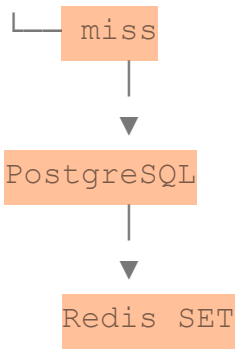
A common technology is Redis.

But caching introduces:

- invalidation
- stale data
- cache stampede
- consistency concerns

A useful pattern is **cache-aside**:



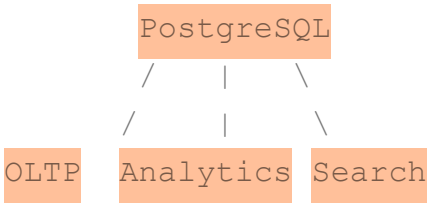


---

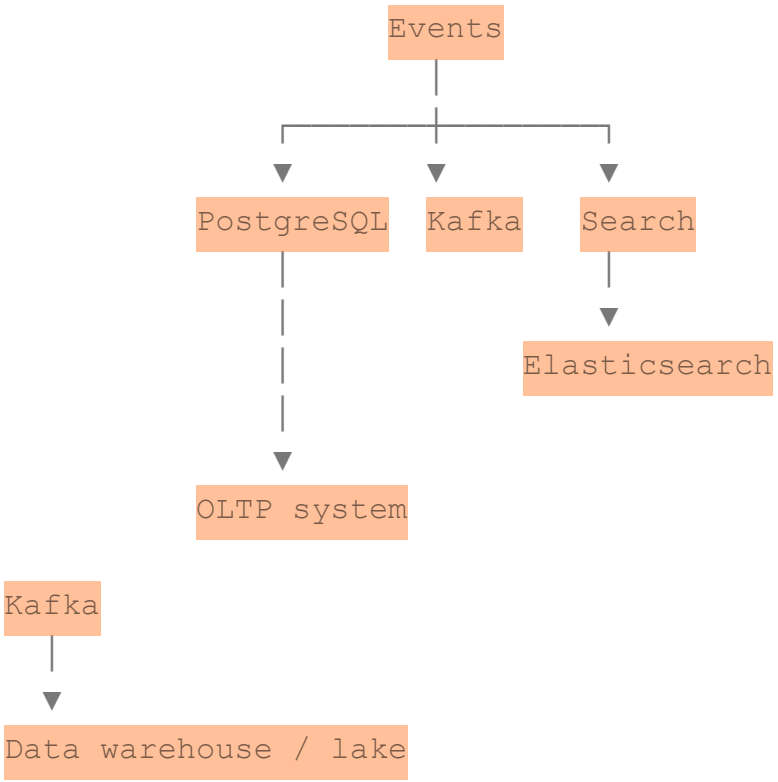
30. Don't use PostgreSQL for everything

At large scale, separate workloads.

Instead of:



consider:

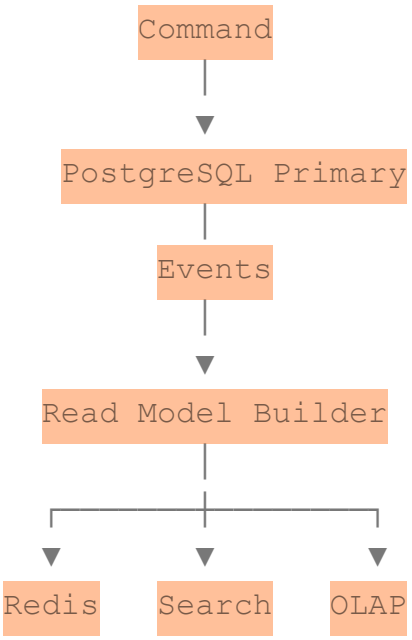


PostgreSQL remains the transactional source of truth.

---

31. CQRS can help

For heavily divergent read/write workloads:



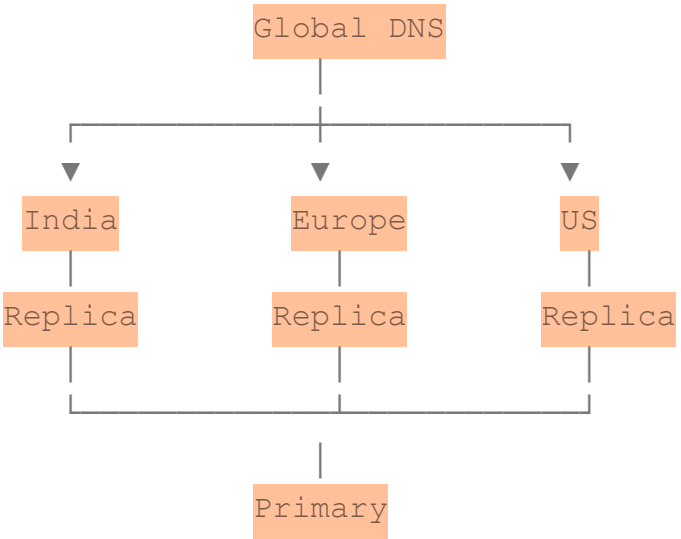
Now the read model is optimized for the actual query.

This is particularly useful when the read workload has fundamentally different requirements from OLTP.

---

32. Multi-region architecture

If the application is global:



But multi-region PostgreSQL introduces difficult questions:

- write locality
- replication latency
- conflict resolution
- failover
- RPO/RTO
- network partitions
- consistency

A common architecture is:

```
Single write region
+
multiple read regions
```

rather than active-active multi-master writes.

---

33. PostgreSQL-specific bottlenecks to monitor

For system design, know these metrics.

Database

- CPU utilization
- memory utilization
- cache hit ratio
- active connections
- transactions/sec
- queries/sec
- lock contention

Query

- p50/p95/p99 latency
- slow queries
- rows scanned
- rows returned
- buffer hits/reads

Storage

- disk utilization
- IOPS
- throughput
- latency
- table/index growth

WAL

- WAL generation rate
- WAL disk usage
- replication lag
- replica replay rate

Vacuum

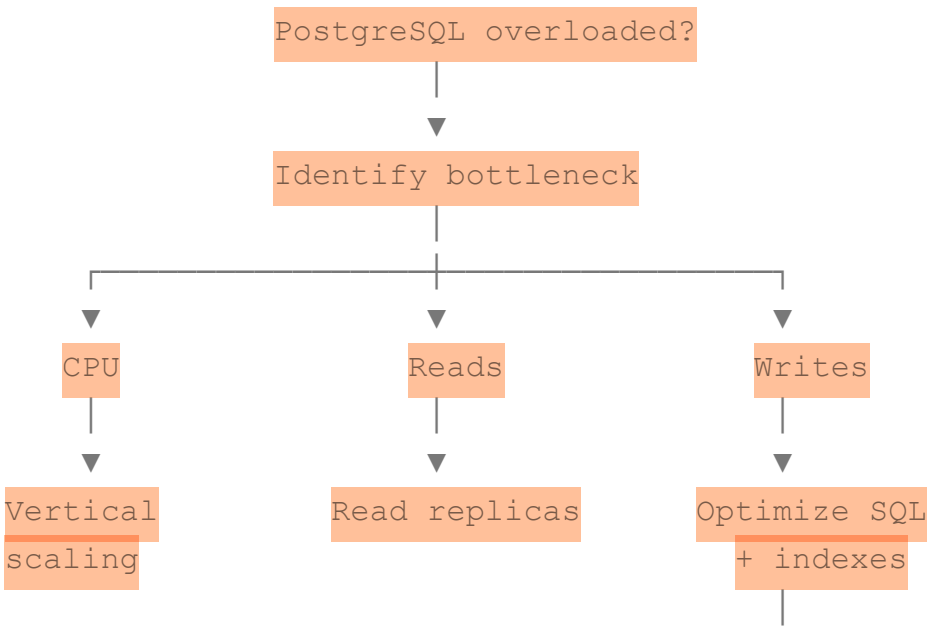
- dead tuples
- autovacuum activity
- table bloat

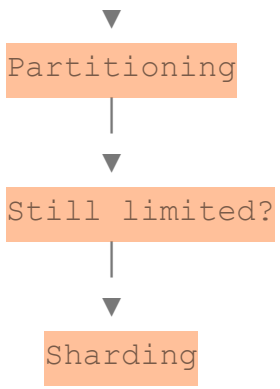
These metrics tell you **which scaling technique to apply next**.

---

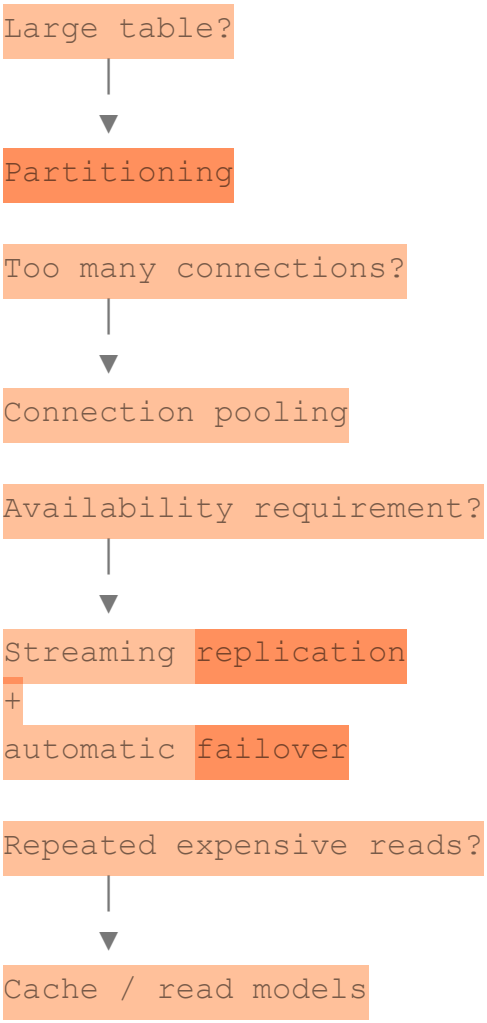
34. A practical scaling decision tree

This is a useful interview framework:





And independently:

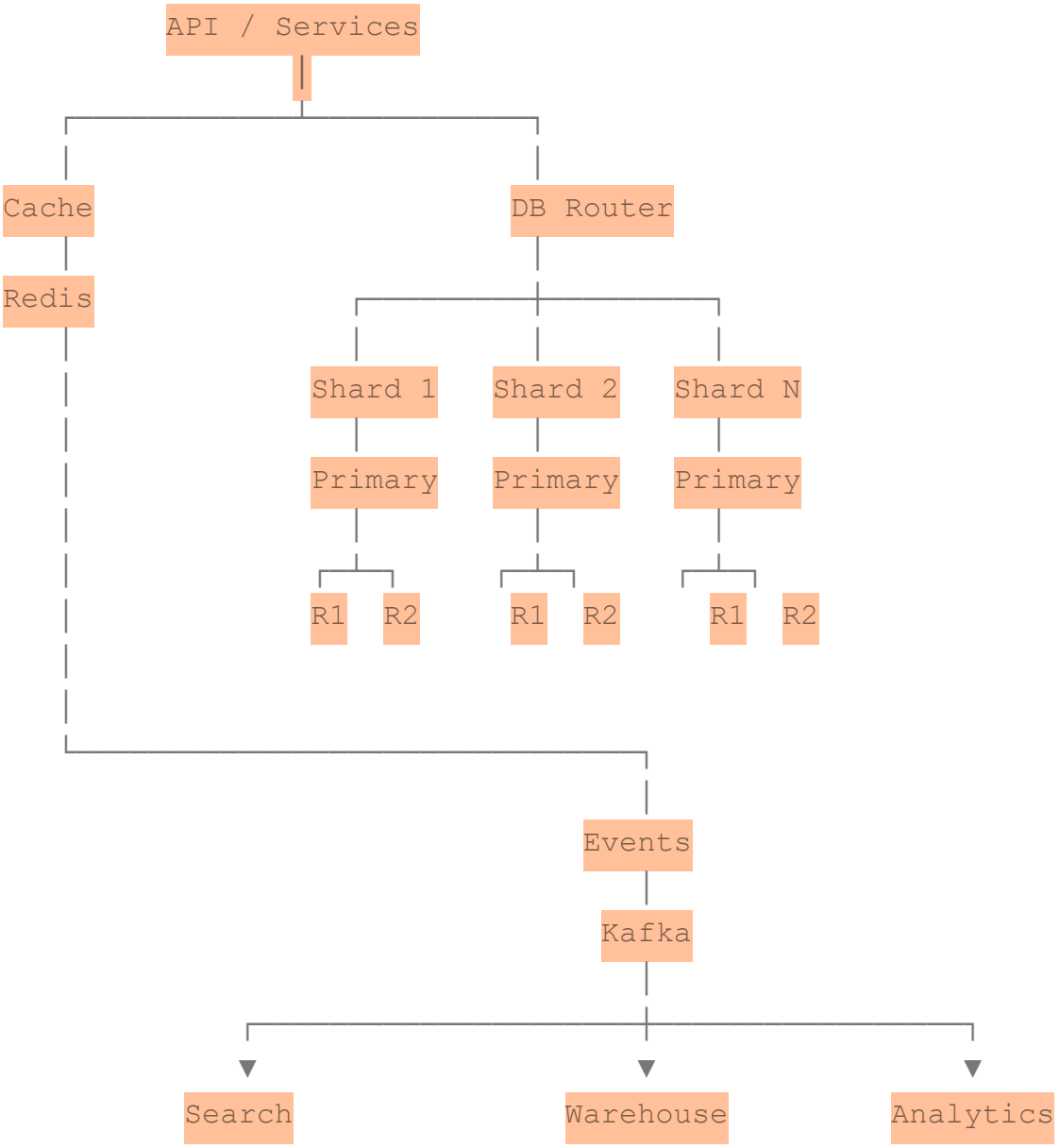


---

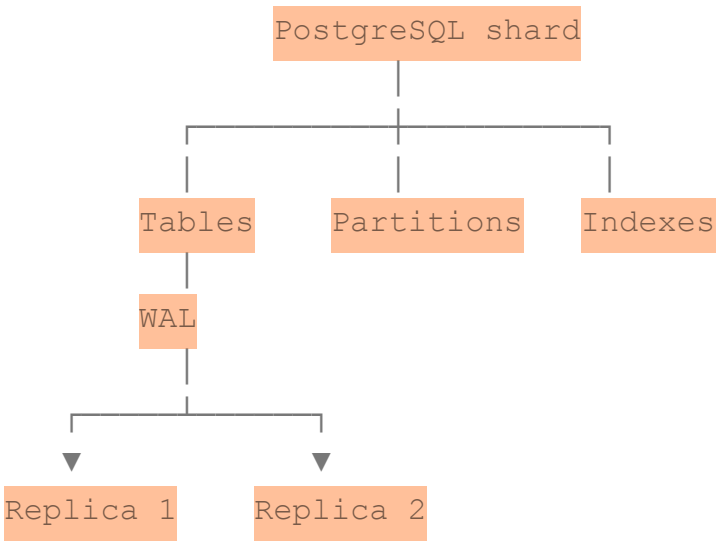
35. A production-grade architecture

For a large OLTP system, you could eventually arrive at something like:





And underneath each PostgreSQL shard:



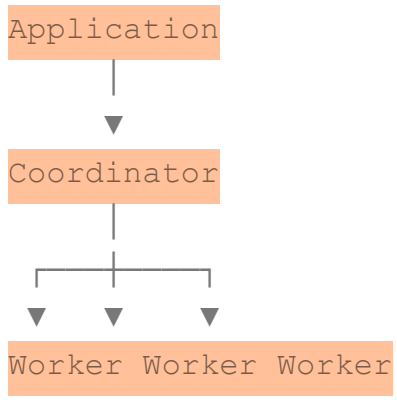
---

36. Where technologies such as Citus fit

You don't necessarily have to implement sharding yourself.

For PostgreSQL-compatible distributed databases, one established option is Citus Data, which extends PostgreSQL for distributed workloads.

The conceptual architecture becomes:



This can simplify:

- distributed tables
- shard placement
- distributed queries
- data distribution

But it introduces distributed-database trade-offs.

In an interview, I'd generally say:

> "If PostgreSQL itself is the requirement, I'd first consider native PostgreSQL scaling techniques. If the workload genuinely requires horizontal write scaling, I'd evaluate a PostgreSQL-compatible distributed solution such as Citus or application-level sharding rather than immediately building a custom distributed database."

---

37. The key trade-offs

Technique	Reads	Writes	Complexity	Main downside
Vertical scaling	↑	↑	Low	Hardware ceiling
Query optimization	↑↑	↑	Low	Requires workload expertise
Indexing	↑↑	↓	Low	Write/storage overhead

Technique	Reads	Writes	Complexity	Main downside
Connection pooling	↑	↑	Low	Pool sizing/management
Read replicas	↑↑↑	–	Medium	Replication lag
Partitioning	↑	↑	Medium	Partition management
Caching	↑↑↑	–	Medium	Invalidation
CQRS/read models	↑↑↑	–	High	Operational complexity
Sharding	↑↑↑	↑↑↑	Very high	Distributed transactions
Multi-region	↑	↑	Very high	Consistency/failover complexity

---

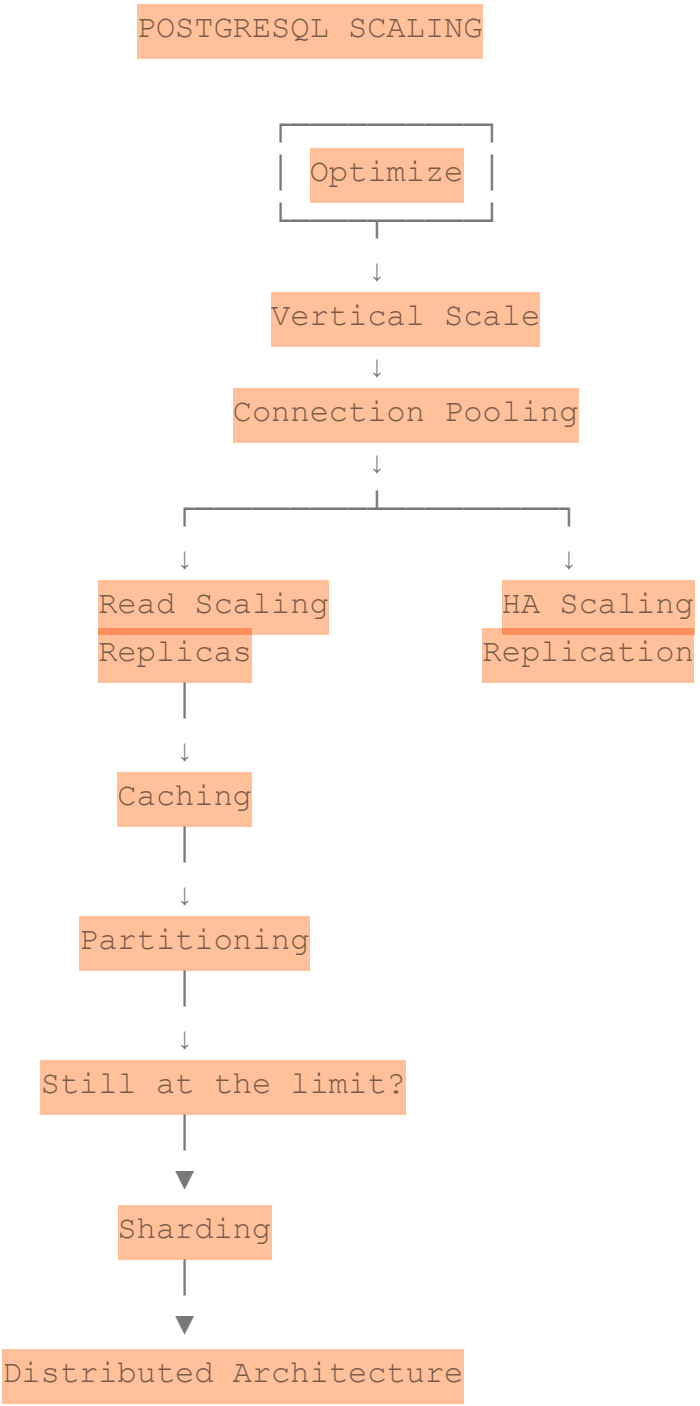
38. What I'd say in a MAANG+ interview

A strong concise answer would be:

> **"I'd scale PostgreSQL incrementally rather than immediately sharding it. First I'd optimize schema, indexes and queries and vertically scale the primary. I'd introduce connection pooling to control connection pressure. For read-heavy workloads, I'd use streaming read replicas and route consistency-sensitive reads to the primary while allowing eventually consistent reads on replicas. I'd use declarative partitioning for very large tables, particularly time-series data, to enable partition pruning and operational lifecycle management. I'd add HA through synchronous/asynchronous replication and automated failover depending on RPO/RTO requirements. If the primary eventually hits its CPU, I/O, WAL or storage ceiling and workload partitioning is possible, I'd shard using a high-cardinality shard key—ideally through logical shards so physical nodes can be rebalanced. I'd explicitly design around cross-shard queries and transactions, using local transactions, outbox/Saga patterns and asynchronous processing where appropriate. Finally, I'd offload caching, search and analytical workloads to systems optimized for those workloads rather than forcing PostgreSQL to handle everything."**

That demonstrates that you understand PostgreSQL not merely as a database, but as a **distributed system component with explicit consistency, availability, partitioning, replication, and operational trade-offs**.

The mental model to remember



The interview principle is: *scale the bottleneck, not the database.*

If you'd like, I can next turn this into a **MAANG+ PostgreSQL system-design interview guide** covering **WAL, MVCC, vacuum, locks, indexes, replication, partitioning, sharding, failover, consistency, and failure scenarios**, with the questions interviewers are most likely to ask.